

Technical Report - Segmentation & Path Planning

Project: AI-Powered Autonomous Drone for Agricultural Survey & Monitoring **Scope:** Phase 1 (Computer Vision / segmentation) and Phase 2 (coverage path planning) **Date:** 2026-06-22

1. Executive Summary

The drone produces an autonomous flight plan over farmland in two stages:

Perception (Phase 1). A U-Net with a ResNet34 encoder reads a drone image and labels **every pixel** as one of five land-cover classes - farmland, building, woodland, water, road. This turns raw imagery into a semantic map of *where the crop is* and *what must be avoided*.

Planning (Phase 2). That map is fed to a coverage path planner that lays down a flight path covering all farmland while routing around hard obstacles (buildings, woodland, water). The output is an ordered list of waypoints plus a mission JSON ready for the rest of the pipeline.

[inference.py](#) is the bridge that runs both stages end to end: image -> mask -> waypoints.

Current model quality: deployed checkpoint reaches **val mIoU 0.8333** (ResNet34, epoch 29); the most recent fine-tune run hit **0.8406**. Per-class IoU: farmland 0.92, water 0.93, woodland 0.90, building 0.81, road 0.64. Target is mIoU ≥ 0.80 , so the model is **over target**, with road remaining the weakest class (thin, rare).

2. Segmentation - U-Net + ResNet34

File: [train.py](#) **Dataset:** LandCover.ai, preprocessed into 256×256 image/mask tiles.

2.1 What it does for the project

Semantic segmentation is the foundation of the whole system. Everything downstream - obstacle maps, the region the drone must cover, coverage statistics, the farmer report - is derived from the per-pixel class map this model produces. Without it there is no notion of "where is the farmland" or "where are the trees."

The five classes map directly onto planning roles:

ID	Class	Role in planning
0	background / farmland	coverage target - the drone must sweep all of it
1	building	hard obstacle
2	woodland	hard obstacle (trees block low-altitude flight)
3	water	hard obstacle
4	road	passable, but not a coverage target

2.2 Why U-Net + ResNet34

- **U-Net** is the standard architecture for segmentation: an encoder down-samples the image to capture context, a decoder up-samples back to full resolution, and **skip connections** carry fine spatial detail from encoder to decoder so object boundaries stay sharp. Critical for thin features like roads.
- **ResNet34 encoder, pretrained on ImageNet.** Using a pretrained backbone gives the model strong low-level features (edges, textures) for free, so it converges faster and generalizes better than training from scratch - important when farmland tiles are visually repetitive. ResNet34 was deliberately chosen over ResNet50: a previous ResNet50 attempt did not justify the extra cost, and the deployed checkpoint is ResNet34 (confirmed by its 278-key state dict).
- Built via `segmentation_models_pytorch` (`smp.Unet`), so the architecture is two lines of code and the encoder swap is a single argument.

An optional **SCSE decoder attention** (`--decoder_attention_scse`) can be enabled to let the decoder re-weight thin features like road; it changes the architecture, so it requires a fresh train.

2.3 The central challenge: class imbalance

The training pixels are wildly imbalanced:

farmland (background)	: 72.3 %
woodland	: 20.6 %
water	: 4.1 %
road	: 1.9 %
building	: 1.1 %

A naive model can score well just by predicting "farmland" everywhere while completely failing on the rare classes that matter most for obstacle avoidance. The training script attacks this on **four independent fronts**:

1. **Weighted combined loss** ([CombinedLoss](#)). Four loss terms, each covering a different weakness: - **Weighted Cross-Entropy** - per-class penalty scaling. Class weights `[0.15, 4.0, 0.8, 1.5, 4.0]` (building and road weighted highest, farmland lowest) directly penalize mistakes on rare classes. - **Dice loss** - optimizes region overlap quality, robust to imbalance. - **Focal loss** ($\gamma=2$) - down-weights easy background pixels so gradient focuses on hard road/building pixels. - **Lovász-Softmax** - a convex surrogate that *directly optimizes per-class IoU* (the actual evaluation metric), specifically to help thin/rare classes like road that pixel-averaged losses under-weight.
2. **Rare-class tile oversampling** (`WeightedRandomSampler`). Tiles that contain building or road pixels are sampled more often (weight `1 + 3.0` per rare class present), so the model sees them more during training. Weights are cached to disk since masks don't change between runs.
3. **Heavy data augmentation** (Albumentations): flips, 90° rotations, brightness/contrast, hue/saturation, grid distortion, Gaussian blur - expands effective dataset variety and prevents overfitting on repetitive farmland.
4. **Weight EMA** ([ModelEMA](#)). An exponential moving average (decay 0.999) of the weights is what gets validated and checkpointed. The averaged weights are smoother, generalize slightly better, and damp the epoch-to-epoch oscillation seen on the rare classes.

2.4 Training mechanics

- **Optimizer**: AdamW (lr 1e-4, weight decay 1e-4).
- **LR schedule**: `ReduceLROnPlateau` on val mIoU by default (halves LR when mIoU stalls). `cosine` and `warmrestart` are also available; `plateau/cosine` were preferred because warm restarts periodically reset the LR and re-flattened mIoU.
- **Mixed precision** (AMP) on CUDA for speed/memory.
- **Metric**: per-class IoU + mean IoU, computed by an `IoUMeter` that accumulates intersection/union across batches.
- **Checkpointing**: saves both the deployable (EMA) weights *and* raw training weights, so a run can resume seamlessly. `--finetune` reloads weights but starts a fresh optimizer + LR schedule at epoch 0, carrying over `best_miou` so a worse fine-tune never overwrites a better checkpoint.
- **Robust resume logic** re-seeds the plateau scheduler's internal "best" from the checkpoint, fixing a subtle bug where the LR would never anneal after a resume.

2.5 Inference

File: [inference.py](#)

Real drone images are far larger than 256×256, so inference uses **tiled sliding-window prediction** (predict):

- Image is reflect-padded and swept with overlapping tiles (tile 256, stride 192).
- **Softmax probabilities** from overlapping tiles are *averaged* before the final `argmax` - this removes the seam artifacts you get from hard-stitching tile boundaries.
- Normalization uses the exact training mean/std, so train and inference are consistent.

Outputs per image: class-ID mask, colour visualization, farmland ROI map, dilated obstacle map, and a coverage-stats JSON (pixel count + % per class). The mask then flows directly into Phase 2.

3. Path Planning - Coverage Path Planners

File: [path_planner.py](#)

3.1 What it does for the project

Given the segmentation mask, the planner answers: *"What flight path makes the drone fly a camera over every part of the farmland, efficiently, without hitting anything?"* This is a **Coverage Path Planning (CPP)** problem (cover an area) combined with **obstacle avoidance** (route around hazards) - distinct from ordinary point-to-point navigation.

It outputs an ordered list of waypoints. Each waypoint carries its pixel position, which sweep line it belongs to, and an `is_transit` flag distinguishing **coverage moves** (camera actively surveying) from **repositioning moves** (flying between coverage runs). `inference.py` then enriches each with heading and cumulative distance and writes a mission JSON.

Two land-cover roles drive the planner: - **Farmland (class 0)** = the region to cover. - **Hard obstacles (building, woodland, water)** = areas to avoid, **dilated by a safety margin** (default 10 px) so the drone keeps clearance. - Road is passable but is not covered.

3.2 The three planners

The file implements three planners with a shared `Waypoint/visualize` interface.

A. RotatingCalipersPlanner (RCPP)

The core idea is **boustrophedon** ("as the ox plows") coverage - back-and-forth parallel sweep lines, like mowing a lawn. The key question is: *which direction should the sweeps run?* Sweeping along the field's **long axis** minimizes the number of expensive U-turns.

- It finds the **minimum-area bounding rectangle** of the farmland polygon via `cv2.minAreaRect`, which internally runs the **Rotating Calipers** algorithm on the convex hull. The rectangle's long axis gives the optimal sweep angle.
- The field is rotated so that axis is horizontal, horizontal sweep lines are drawn `swath_width × (1 - over_lap)` pixels apart, traversable segments on each line are found (farmland AND not obstacle), direction alternates each line, then waypoints are rotated back to the image frame.
- It covers **every** farmland region above a minimum area (largest first), each with its own optimal angle, stitching disconnected regions together.
- **Best for:** open farmland with sparse obstacles.

B. BECDPlanner (Boustrophedon Exact Cell Decomposition)

For fields broken up by **internal obstacles**, a single sweep direction is poor. BECD instead **decomposes** the free space into obstacle-free cells:

- It sweeps a vertical line left->right across the field, tracking connected intervals per column and detecting **topology events** - where the free space **splits** (an obstacle appears) or **merges** (an obstacle ends).
- Each event closes/opens cells, producing a set of simple rectangular cells that together tile all traversable farmland.
- Each cell is covered with vertical boustrophedon sweeps; cells are then visited in **greedy nearest-endpoint order** and stitched together.
- **Best for:** complex fields with many internal obstacles.

C. HybridPlanner - *recommended default*

The hybrid combines the strengths of both:

1. **BECD decomposes** the field into obstacle-free cells (handles internal obstacles).
2. For **each cell**, **RCPP's rotating-calipers step picks that cell's own optimal sweep angle** and sweeps it (minimizes U-turns per cell instead of forcing axis-aligned sweeps).
3. Cells are ordered greedily and stitched with A* transit routing.

The result is **obstacle-aware (from BECD) AND angle-optimal (from RCPP)** at once. It composes the two base planners rather than duplicating logic, so they share one code path. This is the default in both `path_planner.py` and `inference.py`.

3.3 Obstacle-aware transit routing (A*)

A coverage sweep is only safe if the *moves between* sweeps/cells are also safe. All three planners share two routines:

- **_line_free** - Bresenham-style check of whether the straight line between two points is obstacle-free. If clear, a direct transit is used (cheapest).
- **_astar** - when the straight line is blocked, an **A* search** finds a path around the obstacle. It runs on a

down-sampled obstacle grid (max-pooled, so a cell is blocked if any pixel in it is blocked) for speed, supports 8-connected movement with Euclidean step costs, uses a straight-line-distance heuristic, and has a node-expansion cap (50k) with a straight-line fallback if it's hit.

This is what lets the planner thread coverage around buildings, woodland, and water while keeping the safety margin. Transit waypoints are tagged `is_transit=True` so they can be drawn differently and excluded from coverage accounting.

3.4 Tuning knobs

Parameter	Meaning	Default
swath_width	camera footprint width in pixels	40
overlap	lateral overlap between adjacent sweeps (0-1)	0.20
safety_margin	px to inflate obstacles for clearance	10
astar_step	A* grid down-sample factor (speed vs precision)	8

3.5 Outputs

Per planner run: a path visualization PNG (coverage lines yellow, transits orange, start green, finish orange), a waypoints JSON (with heading + cumulative distance), and a mission JSON (image size, planner, total waypoints, total distance, coverage stats). These feed Phase 3 (reporting) and, after pixel->GPS conversion, the actual flight controller.

4. How the two stages connect

```
drone image
| predict() -- tiled U-Net + ResNet34, prob-averaged
v
class-ID mask (0-4)
| farmland -> coverage target
| building/woodland/water -> obstacles (dilated by safety margin)
v
HybridPlanner.plan(mask)
| BECD decompose -> per-cell RCPP angle + boustrophedon -> A* stitch
v
ordered waypoints -> enrich (heading, distance) -> mission JSON
```

`inference.py` runs this whole chain with a single command:

```
python inference.py --image field.jpg --plan --planner hybrid --out_dir results/
```

5. Status & next steps

- [x] Segmentation trained, **over the mIoU >= 0.80 target** (0.8333 deployed / 0.8406 latest fine-tune). Road IoU (~0.64) is the known weak spot.
- [x] Three coverage planners implemented with shared A* obstacle avoidance; Hybrid is the default.
- [] **Pixel -> GPS (lat/lon) conversion** is the main missing link before waypoints can drive a real flight controller (Phase 2 remaining item).
- [] Phase 3: field stats -> natural-language farmer report (Claude API) -> PDF.

Possible improvements

- **Road class:** try SCSE decoder attention, higher-resolution (512 px) tiles, or further raising the road loss weight / oversampling boost.
- **Planner realism:** account for drone turning radius and battery/range limits when ordering cells; currently

ordering is greedy nearest-endpoint.

- **Validate** the pixel->GPS conversion against known ground-truth coordinates once implemented. ``